Prompt Engineering Avanzato

Modulo M2 · Lezione 23



Il primo RAG completo

Stasera si uniscono due gruppi di strumenti costruiti nelle lezioni scorse:

`capire e cercare il testo`

L19 **embeddings** · L20 **search** a mano · L21 **vector DB** · L22 **chunking** dei documenti lunghi

`parlare col modello`

L13 **API** · L14 **token e costi** · L15 **llm_client** e prompt-as-function

Stasera

query → vector DB → chunk nel prompt → **risposta fondata sui documenti, con citazioni** — e il rifiuto pulito quando la risposta non c'è.

Il nuovo della serata è **UNA funzione: il prompt**. Tutto il resto è `aikit/` che avete già costruito.

> Da stasera anche `llm_client` (L15) vive in `aikit/` : il toolkit copre tutta la pipeline.



Agenda

Quando c'è da lavorare, una **slide di switch** indica file e durata; finita l'attività si torna alle slide.
🎤 = il docente scrive, voi replicate · ⌨️ = scrivete voi (docente in affiancamento)

	Blocco	Dove si lavora	Tempo
📊	Teoria: system prompt, struttura, few-shot, CoT, il prompt RAG	slide	50′
🎤	**Live coding: il retrieval**	`working/rag.py`	15′
⌨️	**Esercizio 1: costruisci_prompt e rispondi**	`working/rag.py`	35′
⌨️	**Esercizio 2: il SYSTEM_PROMPT**	`working/rag.py`	15′
📊	Wrap-up + ponte alla L24	slide	5′

La chiave è **quella di L13–L22**. Serve la collection `lumen_chunks` di sabato scorso: chi non ce l'ha la **rigenera con un comando** (lo dice la slide di switch). Costo dell'intera serata: **sotto il centesimo**.

Setup (venv, requirements, `.env`): passo-passo in `scripts/CONSEGNA.md` — si fa **nei primi minuti della teoria**.
Tutti i comandi si lanciano **dalla cartella** `scripts/`.



1. System prompt, struttura, few-shot



System prompt: ruolo, vincoli, formato

La stessa domanda, due contratti diversi col modello:

«Sei un assistente»

Risposta lunga, tono da manuale, nessun limite: il modello decide da solo chi è, come parla e quanto scrive — ogni volta in modo diverso.

Il system di Lumen

CHI sei (assistente clienti Lumen) · **COME** parli (cordiale, concreto, max 4 frasi) · **COSA** non fai. Risposta breve, nel tono giusto, ogni volta.

Il system prompt è il contratto

Si scrive una volta, governa **ogni** risposta. Lo riscrivete voi nell'esercizio 2.

> Estende il **role prompting** visto a L11.



Le tecniche di stasera

Oltre al system prompt, stasera usiamo queste tecniche — una slide ciascuna, in quest'ordine:

`struttura e delimitatori`	istruzioni, dati e domanda in sezioni marcate, per non farle mischiare
`few-shot`	esempi dentro il prompt, per mostrare un formato o uno stile invece di descriverlo
`chain-of-thought`	chiedere al modello di scrivere i passaggi del ragionamento
`self-consistency · ToT`	varianti nate nei paper: più risposte o rami in parallelo — solo panorama
`prompt RAG`	documenti recuperati nel prompt + regole di grounding: il punto d'arrivo, dove le tecniche si combinano



Struttura e delimitatori

Istruzioni, contesto e dati in sezioni **marcate**: il modello non deve indovinare dove finisce una cosa e inizia l'altra.

```
Rispondi alla domanda usando SOLO i documenti qui sotto…

<documenti>
  …i chunk recuperati, ognuno con la sua fonte…
</documenti>

<domanda>
  La centralina mostra il codice E07: cosa devo fare?
</domanda>
```

Costo zero, più robustezza

Testo dell'utente e istruzioni **non si mischiano**: ognuno nella sua sezione.

Si usa nell'esercizio 1

È l'anatomia che diventa il **vostro prompt RAG** nell'esercizio 1.

> Tag XML o intestazioni markdown pari sono: conta la **costanza**, non la sintassi.



Few-shot: mostrare, quando descrivere non basta

La stessa richiesta di formato (ticket a 3 righe: ESITO / AZIONE / FONTE), fatta in due modi — misurata:

`un paragrafo di istruzioni chiare`

Formato **rispettato** — 38 token di risposta.

`2 esempi nel prompt (few-shot)`

Formato **rispettato** — 60 token di risposta.

Quando servono gli esempi

Sui modelli attuali **l'istruzione chiara basta spesso**. Gli esempi si aggiungono quando il formato o lo stile è più facile da **mostrare** che da descrivere — o il modello ignora la descrizione.

Ogni esempio sta nel prompt **a ogni chiamata**: si paga in token, per sempre (L14).

> Nell'esercizio 2 potete definire il formato delle citazioni con 2 esempi nel system.



2. Ragionare costa



Chain-of-Thought, coi numeri

Stessa domanda (le date di un reso in saldo), stesso contesto giusto — con e senza «ragiona passo passo»:

	Risposta	Output	Costo
senza CoT	corretta	79 token	$0.000315
con CoT	corretta, **coi passaggi visibili**	191 token	$0.000499

Cosa aggiunge il CoT, qui

Trasparenza del ragionamento — utile per il debug — non correttezza.

Reasoning model

Pensano prima di rispondere e costano di più: si usano quando il problema lo richiede (mentalità L14).

> Su domande semplici il CoT **allunga la risposta e basta**: provatelo sui vostri casi prima di adottarlo.



Self-consistency e Tree-of-Thought

Due tecniche che incontrerete negli articoli:

`self-consistency`

La stessa domanda **n volte** in parallelo, poi il voto di maggioranza tra le risposte.

`Tree-of-Thought`

Il ragionamento esplorato come un **albero**: più strade, valutate e potate.

Nei paper funzionano. In produzione non si usano.

Costo **×n a ogni domanda**, latenza moltiplicata — e i **reasoning model** e i pattern agentici (→ modulo 4) fanno lo stesso lavoro, meglio.



3. Il prompt RAG



Anatomia del prompt RAG

```
ISTRUZIONI
Rispondi SOLO dai documenti. Cita la fonte.
Se la risposta non c'è: dillo, senza inventare.

<documenti>
  <documento fonte="manuale-lumadesk-007">…</documento>
  <documento fonte="policy-resi-garanzia-002">…</documento>
</documenti>

<domanda> … </domanda>
```

Senza grounding

il modello risponde COMUNQUE, col tono di chi sa: la sicurezza **non è** un segnale di verità.

Il «non c'è» è una risposta GIUSTA

Un RAG che rifiuta vale più di uno che inventa.

> La **fonte sul chunk** è ciò che rende la risposta verificabile: senza, il cliente deve fidarsi.



La pipeline completa

Il disegno che tra poco trasformate in codice — tre funzioni, un file:



`cerca_chunk`

la collection di L22, via `aikit.vectorstore` — **col docente**

`costruisci_prompt`

l'anatomia della slide precedente — **VOI**

`rispondi`

il giro completo, chiuso da `llm_client` (aikit) — **VOI**

> La domanda della serata è la **E07 di sabato scorso**: a L22 tornava il chunk giusto — stasera diventa una **risposta**.

🎤 Live coding — il retrieval

🎤 **LIVE CODING — si guarda e si replica**

`working/rag.py` · 15 minuti

poi il file resta a voi

Prima mossa — la collection di sabato

Chi ha la propria `chroma/` di L22 la copia in `scripts/` ; per gli altri la **riserva** (identica, ~un decimo di centesimo):

```
python scaffolding/prepara_collection.py
```

`TODO 1 — l'unico che scriviamo insieme`

`cerca_chunk(query, k)`

la collection `lumen_chunks` interrogata via `aikit.vectorstore` . **Fatto quando**: il docente la prova a schermo — 4 chunk con score per la E07 — e la vostra fa lo stesso.

In chiusura, la demo col modello DA SOLO (fornita: `rispondi_senza_documenti`)

La E07 senza documenti: il modello **inventa una procedura** plausibile ma sbagliata — sicurissimo. **Tenete a mente quella risposta**: tra mezz'ora la confrontate con la vostra.

⌨ Esercizio 1 — costruisci_prompt e rispondi

⌨ **TOCCA A VOI — docente in affiancamento**

`working/rag.py` · 35 minuti

si torna alle slide
per il confronto

`TODO 2 · costruisci_prompt(query, chunks)`

L'anatomia vista in teoria, **con parole vostre**: sezioni marcate, grounding («SOLO dai documenti»), fonte citata, rifiuto esplicito quando la risposta non c'è.

`TODO 3 · rispondi(query, k)`

Il giro completo: `cerca_chunk` → `costruisci_prompt` → `llm.chat( … , instructions=SYSTEM_PROMPT)`.

Cosa è prescritto

Solo i **nomi**, le variabili del main (`DOMANDA_NEI_DOCUMENTI` , `DOMANDA_FUORI` , `K`) e lo scopo: il come — e le **parole del prompt** — sono vostri.

Fatto quando

La **E07** risponde con la **procedura vera e la fonte** (non quella inventata di prima); il **contrassegno** ottiene il **rifiuto pulito**, non metodi di pagamento inventati. Una risposta ≈ **$0.0004**: provate quanto volete.



📊 Al rientro — con e senza documenti, fianco a fianco

Stesso modello, stesse domande. La differenza è il **prompt**:

Domanda	Senza documenti	Col VOSTRO RAG
«codice **E07**: cosa devo fare?»	procedura **inventata**, tono sicuro	la procedura del **manuale**, con `[manuale-lumadesk-…]`
«posso pagare in **contrassegno**?»	**inventa** i metodi di pagamento accettati	«Non lo trovo nei documenti: ti metto in contatto con un operatore.»

La differenza non è il modello

È lo **stesso**. È il prompt: grounding + i documenti giusti nel contesto.

La risposta inventata era credibile

Plausibile, cortese, ben scritta: senza la versione fondata accanto, **non l'avreste riconosciuta**.

> Le vostre risposte differiranno nelle parole (temperature, prompt vostri): conta il **comportamento**, non la frase.



⌨ Esercizio 2 — il SYSTEM_PROMPT

⌨

TOCCA A VOI — docente in affiancamento

`SYSTEM_PROMPT`, in cima a `working/rag.py` · 15 minuti

si torna alle slide
per il debrief

`TODO 4 · SYSTEM_PROMPT`

Riscrivete il `SYSTEM_PROMPT`: **CHI** è l'assistente · il **TONO** · il **FORMATO** delle citazioni (anche con 2 esempi few-shot, se mostrare vi viene meglio che descrivere) · come deve **SUONARE** il «non c'è».

Fatto quando

Le stesse due domande, **prima e dopo**: la risposta cambia **nel modo che avevate deciso** — e il rifiuto resta pulito.



📊 Al rientro — cosa cambia un system fatto bene

La E07, stessa risposta fondata — due voci diverse:

`system minimo`

«Il codice E07 indica una perdita di calibrazione. Scollega l'alimentazione per 30 secondi, poi tieni premuto il tasto "-"...» — corretta, ma **anonima**: lunghezza e formato decisi dal modello.

`system di Lumen (ruolo · tono · max 4 frasi · formato fonte)`

Stessa sostanza, **voce del brand**: cordiale e concreta, quattro frasi, la fonte nel formato che avete deciso voi.

Due leve, un comportamento

Il prompt RAG **fonda** la risposta (grounding, fonti, rifiuto); il system le dà la **voce** (ruolo, tono, formato). Si progettano insieme.

> Sabato questi due pezzi diventano le **varianti dell'A/B test**.



Cosa portate a casa

🧰

`rag.py` — scritto da VOI

il primo **RAG end-to-end** del corso: retrieval (aikit) + il vostro prompt + il vostro system. Da sabato, il banco di prova del Lab.

🧠

L'anatomia del prompt RAG

istruzioni marcate · documenti con la fonte · domanda — **grounding e rifiuto**, sempre. Il «non c'è» è una risposta giusta.

⚖️

Tre regole pratiche

il grounding si **impone**, non si spera · esempi quando mostrare batte descrivere · ragionare **costa**: CoT = trasparenza, reasoning model quando serve.

Sabato (L24): si misura

Un assaggio: una domanda sui saldi con una query vaga pesca il **chunk sbagliato** → «30 giorni», sbagliato — con e senza CoT. Quando il RAG sbaglia, spesso è il **retrieval**, non il prompt: sabato, **A/B di prompt e tuning del retrieval** sul VOSTRO `rag.py`.

Non cancellate niente: collection e `rag.py` sabato si riusano. (A L30 il RAG avrà la memoria: stasera risponde a domande secche.)



Grazie

L24 · Lab Prompt & Search · sabato 18 luglio

